

Norvig's Probabilistic Spelling Corrector

Diego Besada Rodríguez

[\[My code\]](#) [\[Original Norvig\]](#)

September 2026

1 Bayesian Inference Framework

The spelling corrector approaches error correction through a Bayesian inference framework. The variables involved are:

- w : the potentially erroneous string (e.g., "speling").
- c : a candidate word from the reference vocabulary V (e.g., "spelling").

In this formulation, $P(c | w)$ represents the posterior probability that the candidate c was the intended word given that we observed the string w . The objective is to determine the candidate $\hat{c}$ that maximizes this probability. This is formulated via Bayes' theorem:

$$\hat{c} = \arg \max_{c \in V} P(c | w) = \arg \max_{c \in V} \frac{P(w | c)P(c)}{P(w)} = \arg \max_{c \in V} P(w | c)P(c) \quad (1)$$

Direct estimation of $P(c | w)$ is not possible as the exact thought process of the author is unobservable. Bayes' theorem decomposes this problem into two components removing the evidence denominator $P(w)$, which remains constant across all candidate comparisons for the observed input w :

1. **The Prior Probability** $P(c)$ quantifies how common word c is within the corpus, independent of any typographic error. We estimate this using the maximum likelihood estimate (MLE) over a corpus (`big.txt`):

$$P_{\text{MLE}}(c) = \frac{\text{count}(c)}{N} \quad (2)$$

where N is the total token count. Because N is fixed, maximizing $P(c)$ is equivalent to comparing raw frequency counts (`corpus[c]`).

2. **The Likelihood** $P(w | c)$ reflects the probability that the user intended to type c but produced w instead. To simplify the problem, Norvig discretizes this term into edit distance tiers:

$$P(w | c)_{\text{dist}=0} \gg P(w | c)_{\text{dist}=1} \gg P(w | c)_{\text{dist}=2} \gg 0 \quad (3)$$

This assignment models the reality of typing: errors of edit distance 1 are much more frequent than errors of distance 2.

1.1 Bridging Theory to Code

In Python, solving Equation (1) is split into two steps:

- **Filtering by Likelihood** $P(w | c)$: `get_candidates` applies the hierarchy in Equation (3) by prioritizing candidates at distance 0, falling back to distance 1, and only checking distance 2 if needed:

```
def get_candidates(word, vocab):  
    return (is_known([word], vocab) or  
            is_known(get_edits(word, 1), vocab) or  
            is_known(get_edits(word, 2), vocab) or  
            [word])
```

- **Selecting by Prior** $P(c)$: Since all retained candidates share the same distance tier, we choose the most common. Following Equation (2), `make_correction` finds the candidate with the highest frequency count:

```
def make_correction(word, corpus):  
    return max(get_candidates(word, set(corpus)), key=corpus.get)
```

2 System Implementation

The algorithm is implemented using four helper functions (adapted from Norvig’s original script):

- **Tokenization** (`get_words`): Extracts lowercase words via regex (`\w+`) and builds a **Counter** mapping tokens to raw frequencies from the corpus.
- **Edit Generation** (`get_edits`): Applies Damerau–Levenshtein operations (deletions, transpositions, replacements, insertions) making edits up to distance n .
- **Vocabulary Filtering** (`is_known`): Removes candidates using set intersection ($W \cap V$) to keep only valid vocabulary words.
- **Final candidates** (`get_candidates`): Evaluates distances hierarchically ($0 \rightarrow 1 \rightarrow 2$), defaulting to $\{w\}$ if no vocabulary match is found.

To quantify performance, there is a function that takes labeled pairs of (**target**, **erroneous**) words. It passes each erroneous word to the corrector, counts a success whenever the predicted output matches the target string, and calculates accuracy as the ratio of correct predictions to total cases.

The model was evaluated on Norvig’s reference test sets using the `big.txt` corpus ($\sim 1.1 \times 10^6$ tokens):

Test Set	Pairs (n)	Accuracy (%)	OOV (%)	Throughput (words/s)
testset1	270	75%	6%	88
testset2	400	68%	11%	78

The lower accuracy on `testset2` (68% vs. 75%) correlates directly with its higher Out-of-Vocabulary (OOV) rate (11% vs. 6%). Because candidates are restricted to known corpus words, unseen words receive zero probability ($P(c) = 0$), forming an upper bound on model accuracy.

3 Limitations and Conclusion

This spelling corrector has several limitations. The most critical one is its lack of context. Context is the primary signal humans rely on to decipher what was intended when a word is misheard or misspelled. A candidate word might have a high frequency in the corpus, but make no sense within the actual sentence; looking at neighboring words would yield much more reliable predictions. Furthermore, while discretizing the likelihood into distance tiers is practical, it remains deficient: it treats all edits equally, ignoring physical keyboard proximity and phonetic similarities.

Nevertheless, the brilliance of the model lies in its simplicity. Unlike modern neural-network-based approaches, Norvig’s model is entirely explainable: each time a correction is made, we can trace exactly why that candidate won: it was simply the most frequent valid word with the shortest available edit distance. Achieving solid performance with negligible computational overhead is what makes this model stand out.